

✓ Approach A: BERT + contextual augmentation (Team 14 / paper replication)

This notebook implements **Approach A** from the midsem deck:

- **Label mapping** (Kaggle Mental Health Text → 3 severity tiers):
 - **Normal** → **No Symptoms** (class 0)
 - **Anxiety**, **Depression** → **Mild/Moderate** (class 1)
 - **Suicidal** → **Severe** (class 2)
- **Model**: **bert-base-uncased** sequence classification.
- **Imbalance**: contextual augmentation for the **Severe** class using BERT masked-language substitutions (Kobayashi-style contextual augmentation, as cited in the base paper).

Colab: Runtime → GPU recommended. Upload **mental_heath_unbanlanced.csv** or mount Google Drive and set **CSV_PATH** below.

```
# Install compatible dependencies (Colab-safe)
```

```
%pip install -q transformers datasets accelerate scikit-learn pandas tqdm sentencepiece
```

```
import json
import os
import random
import re
from pathlib import Path

import numpy as np
import pandas as pd
import torch
from datasets import Dataset
from sklearn.metrics import accuracy_score, classification_report, f1_score, recall_score
from sklearn.model_selection import train_test_split
from tqdm.auto import tqdm
from transformers import (
    AutoModelForSequenceClassification,
    AutoTokenizer,
    Trainer,
    TrainingArguments,
)

# Reproducibility
SEED = 42
random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)
if torch.cuda.is_available():
    torch.cuda.manual_seed_all(SEED)

# --- Paths (edit for Colab) ---
# Example after uploading to Colab:
CSV_PATH = "/content/mental_heath_unbanlanced.csv"
# Or Google Drive:
# from google.colab import drive
# drive.mount("/content/drive")
# CSV_PATH = "/content/drive/MyDrive/.../mental_heath_unbanlanced.csv"

CSV_PATH = os.environ.get("CSV_PATH", "mental_heath_unbanlanced.csv")
OUTPUT_DIR = Path(os.environ.get("OUTPUT_DIR", "approach_a_bert_model"))
OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

MODEL_NAME = "bert-base-uncased"
MAX_LENGTH = 128

# Augmentation: BERT contextual substitutes for Severe class (training split only).
# Full augmentation on all Severe rows can be slow; raise cap for stricter replication.
AUGMENTS_PER_SEVERE_SAMPLE = 1
MAX_SEVERE_SAMPLES_TO_AUGMENT = 4000 # set to None to augment every Severe training example

print("CUDA:", torch.cuda.is_available(), torch.cuda.get_device_name(0) if torch.cuda.is_available() else "")
```

```
CUDA: True Tesla T4
```

1. Load CSV and map labels (PPT specification)

```
df = pd.read_csv("/content/mental_heath_unbanlanced.csv")
assert "text" in df.columns and "status" in df.columns, df.columns.tolist()

RAW_TO_SEVERITY = {
    "Normal": "No Symptoms",
    "Anxiety": "Mild/Moderate",
    "Depression": "Mild/Moderate",
    "Suicidal": "Severe",
}

label2id = {"No Symptoms": 0, "Mild/Moderate": 1, "Severe": 2}
id2label = {v: k for k, v in label2id.items()}

unknown = set(df["status"].unique()) - set(RAW_TO_SEVERITY.keys())
if unknown:
    raise ValueError(f"Unexpected status values: {unknown}")

df["severity_label"] = df["status"].map(RAW_TO_SEVERITY)
df["label"] = df["severity_label"].map(label2id)
df[["status", "severity_label", "label"]].value_counts()
```

		count	
status	severity_label	label	
Normal	No Symptoms	0	18391
Depression	Mild/Moderate	1	14506
Suicidal	Severe	2	11212
Anxiety	Mild/Moderate	1	5503

dtype: int64

2. Text cleaning (light normalization)

```
def clean_text(text: str) -> str:
    text = str(text).strip()
    text = re.sub(r"http\S+", " ", text)
    text = re.sub(r"\s+", " ", text).strip()
    return text

df["text_clean"] = df["text"].map(clean_text)
df = df[df["text_clean"].str.len() > 0].reset_index(drop=True)
df["severity_label"].value_counts()
```

		count
severity_label		
Mild/Moderate		20009
No Symptoms		18391
Severe		11211

dtype: int64

3. Stratified train / validation / test split

```
import os
import pandas as pd

# Check if we already have the saved augmented data
if os.path.exists('train_augmented.csv'):
    print("Loading existing augmented datasets from CSV...")
    df_train_aug = pd.read_csv('train_augmented.csv')
    df_val = pd.read_csv('val.csv')
```

```

df_test = pd.read_csv('test.csv')

X_train_aug, y_train_aug = df_train_aug['text'].tolist(), df_train_aug['label'].tolist()
X_val, y_val = df_val['text'].tolist(), df_val['label'].tolist()
X_test, y_test = df_test['text'].tolist(), df_test['label'].tolist()

print(f"Loaded: Train({len(X_train_aug)}), Val({len(X_val)}), Test({len(X_test)})")
else:
    # Original splitting logic if CSVs don't exist
    X = df["text_clean"].tolist()
    y = df["label"].tolist()

    X_train, X_temp, y_train, y_temp = train_test_split(
        X, y, test_size=0.2, random_state=SEED, stratify=y
    )
    X_val, X_test, y_val, y_test = train_test_split(
        X_temp, y_temp, test_size=0.5, random_state=SEED, stratify=y_temp
    )
    print("Train", len(X_train), "Val", len(X_val), "Test", len(X_test))

```

Train 39688 Val 4961 Test 4962

4. Contextual augmentation (Severe class, train only)

Uses `nlpaug` BERT substitution (contextualized replacements), consistent with the paper's discussion of contextual augmentation / contextualized embeddings for minority classes.

```

if os.path.exists('train_augmented.csv'):
    print("Augmented data already exists. Skipping BERT-based augmentation loop.")
else:
    from transformers import pipeline
    import torch

    print("Initializing augmentation pipeline...")
    aug_device = 0 if torch.cuda.is_available() else -1
    mask_filler = pipeline("fill-mask", model="bert-base-uncased", device=aug_device)

    def augment_severe_texts(texts: list[str], labels: list[int]) -> tuple[list[str], list[int]]:
        severe_idx = [i for i, lab in enumerate(labels) if lab == label2id["Severe"]]
        if MAX_SEVERE_SAMPLES_TO_AUGMENT is not None and len(severe_idx) > MAX_SEVERE_SAMPLES_TO_AUGMENT:
            rng = random.Random(SEED)
            severe_idx = rng.sample(severe_idx, MAX_SEVERE_SAMPLES_TO_AUGMENT)

        new_texts = list(texts)
        new_labels = list(labels)
        severe_label = label2id["Severe"]

        for i in tqdm(severe_idx, desc="BERT-based aug (Severe)"):
            text = texts[i]
            words = text.split()
            if len(words) < 5: continue
            try:
                idx_to_mask = random.randint(0, len(words) - 1)
                original_word = words[idx_to_mask]
                words[idx_to_mask] = "[MASK]"
                masked_sent = " ".join(words)
                predictions = mask_filler(masked_sent)
                for pred in predictions:
                    replacement = pred['token_str'].strip()
                    if replacement.isalpha() and replacement.lower() != original_word.lower():
                        words[idx_to_mask] = replacement
                        break
                augmented_text = clean_text(" ".join(words))
                if augmented_text != text:
                    new_texts.append(augmented_text)
                    new_labels.append(severe_label)
            except Exception: continue
        return new_texts, new_labels

    X_train_aug, y_train_aug = augment_severe_texts(X_train, y_train)
    print("Train after Severe aug:", len(X_train_aug))

```

```

Initializing augmentation pipeline...
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:93: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens)
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_
Loading weights: 100% 202/202 [00:00<00:00, 355.43it/s, Materializing param=cls.predictions.transform.dense.weight]

BertForMaskedLM LOAD REPORT from: bert-base-uncased
Key | Status | |
-----+-----+--
cls.seq_relationship.bias | UNEXPECTED | |
cls.seq_relationship.weight | UNEXPECTED | |
bert.pooler.dense.bias | UNEXPECTED | |
bert.pooler.dense.weight | UNEXPECTED | |

Notes:
- UNEXPECTED :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
BERT-based aug (Severe): 100% 4000/4000 [03:07<00:00, 23.57it/s]
You seem to be using the pipelines sequentially on GPU. In order to maximize efficiency please use a dataset
Token indices sequence length is longer than the specified maximum sequence length for this model (1845 > 512). Runni
Train after Severe aug: 43583

```

5. Tokenize and Hugging Face Dataset

```

tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)

def encode_batch(batch):
    enc = tokenizer(
        batch["text"],
        truncation=True,
        max_length=MAX_LENGTH,
        padding="max_length",
    )
    enc["labels"] = batch["label"]
    return enc

train_ds = Dataset.from_dict({"text": X_train_aug, "label": y_train_aug})
val_ds = Dataset.from_dict({"text": X_val, "label": y_val})
test_ds = Dataset.from_dict({"text": X_test, "label": y_test})

train_ds = train_ds.map(encode_batch, batched=True, remove_columns=["text"])
val_ds = val_ds.map(encode_batch, batched=True, remove_columns=["text"])
test_ds = test_ds.map(encode_batch, batched=True, remove_columns=["text"])

train_ds.set_format(type="torch", columns=["input_ids", "attention_mask", "labels"])
val_ds.set_format(type="torch", columns=["input_ids", "attention_mask", "labels"])
test_ds.set_format(type="torch", columns=["input_ids", "attention_mask", "labels"])

Map: 100% 43583/43583 [00:22<00:00, 1746.35 examples/s]
Map: 100% 4961/4961 [00:03<00:00, 1525.81 examples/s]
Map: 100% 4962/4962 [00:02<00:00, 2157.43 examples/s]

```

6. Fine-tune BERT

```

num_labels = len(label2id)
model = AutoModelForSequenceClassification.from_pretrained(
    MODEL_NAME,
    num_labels=num_labels,
    id2label=id2label,
    label2id=label2id,
)

# Move model to GPU if available
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

```

```
model.to(device)
print(f"Model is training on: {device}")

def compute_metrics(eval_pred):
    logits, labels = eval_pred
    preds = np.argmax(logits, axis=-1)
    acc = accuracy_score(labels, preds)
    macro_f1 = f1_score(labels, preds, average="macro")
    recalls = recall_score(labels, preds, average=None, labels=[0, 1, 2], zero_division=0)
    severe_recall = float(recalls[label2id["Severe"]])
    return {"accuracy": acc, "macro_f1": macro_f1, "recall_severe": severe_recall}

# --- Use Entire Dataset ---
train_ds_fast = train_ds

args = TrainingArguments(
    output_dir=str(OUTPUT_DIR / "checkpoints"),
    learning_rate=2e-5,
    per_device_train_batch_size=16,
    per_device_eval_batch_size=32,
    num_train_epochs=1,
    weight_decay=0.01,
    eval_strategy="steps",
    eval_steps=100,
    save_strategy="no",
    load_best_model_at_end=False,
    seed=SEED,
    report_to="none",
    fp16=torch.cuda.is_available(),
)

trainer = Trainer(
    model=model,
    args=args,
    train_dataset=train_ds_fast,
    eval_dataset=val_ds.select(range(min(500, len(val_ds)))),
    processing_class=tokenizer,
    compute_metrics=compute_metrics,
)

trainer.train()
```

Loading weights: 100%199/199 [00:00<00:00, 464.49it/s, Materializing param=bert.pooler.dense.weight]

BertForSequenceClassification LOAD REPORT from: bert-base-uncased

Key	Status
cls.predictions.transform.dense.bias	UNEXPECTED
cls.predictions.transform.LayerNorm.bias	UNEXPECTED
cls.predictions.bias	UNEXPECTED
cls.predictions.transform.dense.weight	UNEXPECTED
cls.seq_relationship.weight	UNEXPECTED
cls.predictions.transform.LayerNorm.weight	UNEXPECTED
cls.seq_relationship.bias	UNEXPECTED
classifier.weight	MISSING
classifier.bias	MISSING

Notes:

- UNEXPECTED :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- MISSING :those params were newly initialized because missing from the checkpoint. Consider training on your downst

Model is training on: cuda

[2724/2724 11:00, Epoch 1/1]

Step	Training Loss	Validation Loss	Accuracy	Macro F1	Recall	Severe
100	No log	0.539052	0.786000	0.782887		0.863248
200	No log	0.473206	0.800000	0.794478		0.854701
300	No log	0.427600	0.832000	0.817819		0.675214
400	No log	0.447527	0.828000	0.823034		0.914530
500	0.535075	0.397670	0.820000	0.812449		0.794872
600	0.535075	0.364802	0.864000	0.855764		0.760684
700	0.535075	0.366860	0.854000	0.842672		0.735043
800	0.535075	0.384707	0.848000	0.839765		0.752137
900	0.535075	0.385704	0.836000	0.828852		0.803419
1000	0.419003	0.351100	0.854000	0.847913		0.863248
1100	0.419003	0.336493	0.854000	0.846091		0.803419
1200	0.419003	0.343238	0.850000	0.844332		0.863248
1300	0.419003	0.347825	0.854000	0.847319		0.829060
1400	0.419003	0.323336	0.872000	0.865096		0.820513
1500	0.395489	0.327428	0.866000	0.859235		0.820513
1600	0.395489	0.311902	0.870000	0.862769		0.829060
1700	0.395489	0.317189	0.858000	0.847956		0.769231
1800	0.395489	0.332976	0.862000	0.854933		0.846154
1900	0.395489	0.316837	0.866000	0.858166		0.811966
2000	0.362957	0.320842	0.868000	0.859897		0.820513
2100	0.362957	0.328893	0.862000	0.855300		0.846154
2200	0.362957	0.312101	0.872000	0.863267		0.803419
2300	0.362957	0.323547	0.864000	0.857229		0.854701
2400	0.362957	0.319022	0.864000	0.856674		0.829060
2500	0.352178	0.320292	0.870000	0.862562		0.820513
2600	0.352178	0.316655	0.872000	0.864076		0.820513
2700	0.352178	0.318521	0.864000	0.856786		0.837607

TrainOutput(global_step=2724, training_loss=0.40854660739982707, metrics={'train_runtime': 661.026, 'train samples per second': 65.932, 'train steps per second': 4.121, 'total flos': 2866818021209856.0, 'train loss':

7. Test set metrics (macro-F1 + per-class, esp. Severe recall)

preds_output = trainer.predict(test_ds)
y_true = preds_output.label_ids
y_pred = np.argmax(preds_output.predictions, axis=-1)

target_names = [id2label[i] for i in range(num_labels)]

```
print(classification_report(y_true, y_pred, target_names=target_names, digits=4))  
print("Accuracy:", accuracy_score(y_true, y_pred))  
print("Macro-F1:", f1_score(y_true, y_pred, average="macro"))
```

	precision	recall	f1-score	support
No Symptoms	0.9622	0.9261	0.9438	1840
Mild/Moderate	0.8768	0.7721	0.8212	2001
Severe	0.6865	0.8751	0.7694	1121
accuracy			0.8525	4962
macro avg	0.8418	0.8578	0.8448	4962
weighted avg	0.8655	0.8525	0.8549	4962

Accuracy: 0.8524788391777509
Macro-F1: 0.8447826618127007

8. Save model + `label_map.json` for the standalone chatbot